

Assignment 6

1 Instructions

- **Deadline:** 24 hours before next laboratory class.
- **Before implementing**, draw **UML Class Diagrams**, save them as images and **copy each one into the root of their project**.
- Rename project. Export them to a **single** archive file `surname1_surname2_name_lab06`.
- Fill the **checklist** and copy to the root of the project.

Whatever the exercise, be sure you **always meet all the following rules**:

1. Unless other thing is stated, null is considered **illegal argument** as well as empty and blank strings.
2. Write **tests first**. When **updating** a project, check that existing tests keep working properly. Even more, include new characteristics in existing tests.

2 Problem statement

Keeping plants healthy requires the best possible growing environment. But staying on top of all environmental changes and equipment statuses or failures can be a challenge. Therefore, we came up with the idea of building a smart garden, **an application to regulate inner conditions in the greenhouse**. The system will have **sensors**, **actuators** and **controllers**.



Sensors will measure environmental conditions. There are many different sensors in the greenhouse: temperature, moisture, radiation, CO_2 , power supply, access control, etc. to measure conditions.

Controllers will decide if conditions are right or need to be changed.

Actuators are elements in the greenhouse than can change conditions. Some are manual, other are automatic.

Also, we need to **check the status of electronic devices** in the greenhouse.

3 Starting project

Main Creates 10 doors (**actuators**), 10 temperature **sensors** and a single greenhouse **controller**. Then, it starts the controller.

GreenhouseController It is a central control system monitoring other controllers in the garden. These controllers control specific conditions as temperature, moisture, etc.

It has a method **start** () that runs an infinite loop to assess environmental conditions **periodically**. Each iteration, **it displays messages produced by the specific controllers** and waits some seconds before repeating the process.

It also displays messages so the gardener can read them.

TemperatureController Described below in section 4.3.

ManualDoor Each door is fully identified. There is also a field (**status**) stating whether it is opened (**true**) or closed (**false**). Methods **open** and **close** will open or close doors respectively by changing the status. Then, it returns one of the following message **Please, open door id** or **Please, close door id** to be printed in the central panel.

TemperatureSensor Class **TemperatureSensor** has an identifier and a method **int readValue()** that returns temperature measured by the sensor.

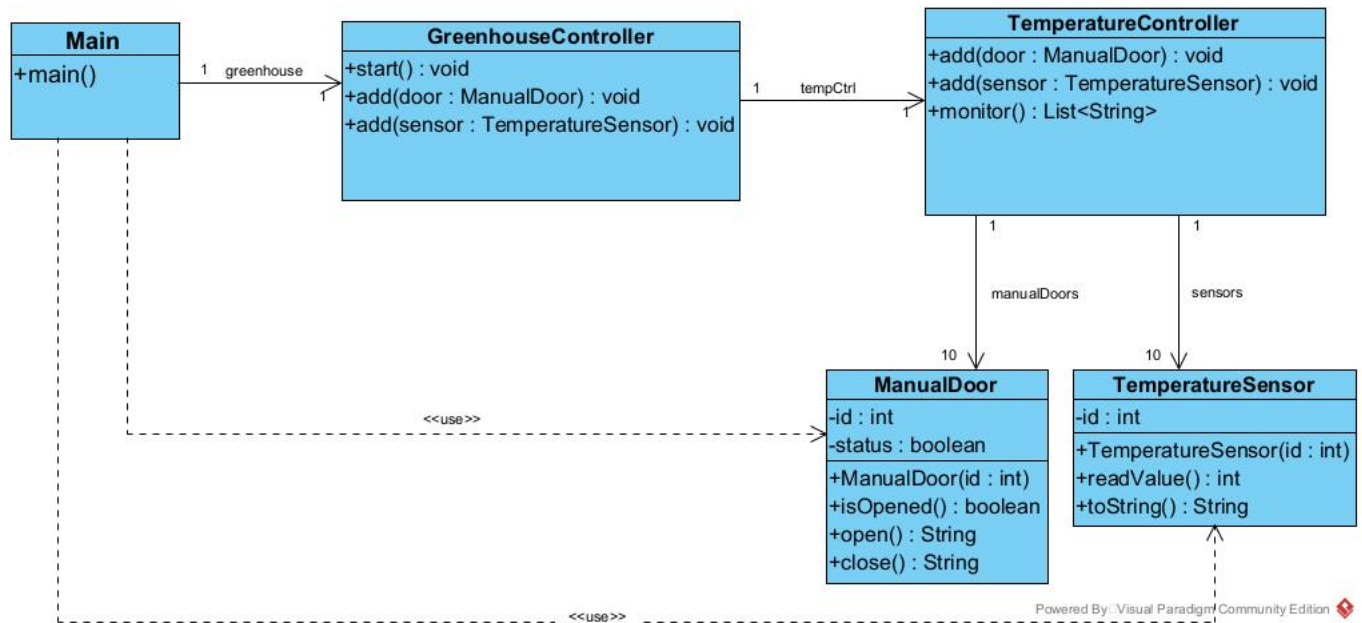


Figure 1: Dependencies not displayed

4 Exercise 1: Implement a minimum working version of the greenhouse

It is implemented by several **temperature sensors**, several **doors** and one **temperature controller**.

4.1 Write unit tests

First, write unit tests for the method **TemperatureController::add(TemperatureSensor sensor)**. Then, implement the method.

4.2 Temperature Sensors

They can be read by invoking **int readValue()** that returns indoor temperature right now. This operation is simulated returning a random number between 5 and 40 degrees.



4.3 Temperature controller

It has a list of temperature **sensors** and a list of doors (**actuators**) and it maintains temperature inside the garden in the growing range by gathering data from sensors periodically, calculating average temperature and, if that value falls outside the threshold range (**minimum limit** 19 degrees, **maximum limit** 22 degrees), **deciding whether it is necessary to open or close doors to regulate temperature**. It returns a list of Strings to the central panel.

Implement `TemperatureController::monitor()` so that

1. Calculate **average temperature between all the sensors**.
2. If it is within the **safe range**, just inform the gardener that needs to do nothing.

`Current temp is <t>. It is right`

3. If it exceeds upper threshold (**it is too hot**), adds this message to the messages returned

`Current temp is <t>. It is too hot`

and decides what it is to be done to cool down.

4. When it is below the lower threshold (**it is too cold**), adds this message to the messages returned

`Current temp is <t>. It is too cold`

and decides what is to be done to warm up.

4.4 To warm up (or cold down)

Let's assume is colder outside. Then, temperature controller decides that the best thing that can be done **to warm up the room is to close doors**.

- Calculate how many doors must be closed (**10% of the doors for each degree under minimum or fraction**) and inform the gardener by sending to `GreenhouseController` to display

`x doors must be closed`

Then, to simulate gardener operation closing door,

`Please, close door y`

- When more doors should be closed but there is no open door,

`WARNING: Can not close enough doors. D more doors should be closed`

5 Exercise 2: Another door in the wall

Ten new doors will be opened in the wall of the garden. This time we install automatic doors that operate without human intervention. The messages will be different. Instead of

`Please, open door x it is Automatic door <id> is opening`

`Please, close door x it is Automatic door <id> is closing`

6 Device scanner

To ensure the proper functioning of the electronic devices, there is a device scanner in the greenhouse. It checks if all them are working properly, reporting the failure of damaged sensors by returning a message to inform the gardener. This check is also done periodically.

6.1 Implement a working version of DeviceScanner

DeviceScanner controls all electronic devices, temperature sensors, moisture sensors, and automatic doors, and reports the failing ones.

- Contains a list of electronic devices to be controlled. Every one will implement a method `boolean check()` that returns true if the device is working properly, false if it is failing. Let's consider that devices work properly with a probability .95.
- Method `List<String> monitor ( )` checks device status from time to time and returns messages to inform the gardener about the operation.
- DeviceScanner must also be invoked in `GreenhouseController::start()`
- If any device fails, `DeviceScanner::scan` returns warning messages

WARNING: <device toString()> is not in a good condition.

Where `toString` must return `[TemperatureSensor]`, `[AutomaticDoor]` or `[MoistureSensor]` plus de id.

7 Moisture control

The greenhouse is also equipped with sensors, actuators and controllers to maintain average moisture within a comfort zone between 60% and 80%.

Moisture sensor Class `MoistureSensor` has an identifier and a method `int readValue()` that returns moisture measured by the sensor as a (random) integer value between 40% and 100%.

Moisture controller Moisture controller reads values from sensors using `int readValue()`, calculates the average moisture in the greenhouse and, if value is outside growing range, it adjusts the irrigation dose by acting on the irrigation system.

Irrigation equipment It is intended to regulate the amount of water applied to land. It is a four position rotary switch (OFF, LOW, MEDIUM and HIGH) and the initial value is MEDIUM.





7.1 Moisture Controller

It has a list moisture sensors and a irrigation equipment. The core of this controller is implemented in the method `List<String> monitor()` that works as follow:

1. Calculates average moisture
2. If it is in the **comfort range**, return:

`Current moisture is <value here>. It is right`

3. If it is **too high** (20% over upper limit), set OFF irrigation system and inform the gardener
`Current moisture is <value here>. It is too damp.`
`Irrigator system is <original value> and is set to <new value here>`
4. If it too low (20% below lower limit) then, set irrigation to HIGH (maximum position) and inform the gardener
`Current moisture is <value here>. It is too dry.`
`Irrigator system is <original value> and is set to <new value here>`
5. Otherwise, just turn the irrigation system up or down one level and return messages
`Current moisture is <value here>. It is a bit too <damp/dry>.`
`Irrigator system is <original value> and is set to <new value here>`

8 Start up

Create a greenhouse with 10 manual doors, 10 automatic doors, 10 temperature sensors, and 10 moisture sensors. Temperature and moisture are monitored on regular basis. Scanner also checks correct operation of electronic devices (moisture sensors) with the same frequency.

9 Sample output

The following text corresponds with an execution of the greenhouse. Please consider only the format, as values and order of operations will probably be different.

```
Current temp is 26.8. It is too hot
10 doors must be opened
Please, open door 6
Automatic door 6 is opening
Please, open door 7
Automatic door 7 is opening
Please, open door 8
Automatic door 8 is opening
Please, open door 9
Automatic door 9 is opening
WARNING: Can not open enough doors. 2 more doors should be opened
WARNING: [MoistureSensor] 6 is not in good condition
Current moisture is 76.1. It is right
Current temp is 17.1. It is too cold
4 doors must be closed
```



```
Please, close door0
Automatic door 0 is closing
Please, close door1
Automatic door 1 is closing
WARNING: [AutomaticDoor] 1 is not in good condition
Current moisture is 85,40 . It is a bit too damp.
Irrigator system is OFF and is set to OFF
Current temp is 23.0. It is too hot
2 doors must be opened
Please, open door 0
Automatic door 0 is opening
WARNING: [MoistureSensor] 0 is not in good condition
Current moisture is 73.0. It is right
Current temp is 23.6. It is too hot
4 doors must be opened
Please, open door 1
Automatic door 1 is opening
WARNING: Can not open enough doors. 2 more doors should be opened
WARNING: [TemperatureSensor] 5 is not in good condition
Current moisture is 66.7. It is right
```

